

Software Test Plan (STP)

Simple Shell (Command Line Interpreter)

Project: Simple Shell (Command Line Interpreter)

Version: 1.0

Authors: Aarushi Singh (PES2UG24CS015), Akshaya M (PES2UG24CS050), Akshay Anil Kumar (PES2UG24CS049),
Adarsh Shivanand Madli (PES2UG24CS028)

Date: 16-09-2026

Status: Draft / For Review

1. Introduction

Purpose: This document defines the test plan for the Simple Shell (Command Line Interpreter) project, v1.0. It outlines the objectives, scope, test strategy, resources, schedule, and responsibilities for verifying that the shell meets the functional, non-functional, and security requirements defined in the project's Software Requirements Specification (SRS).

Scope: Testing covers command input and parsing, built-in commands (cd, pwd, echo, help, exit, history), external command execution via PATH lookup and fork/exec, foreground and background process management, input/output redirection (<, >, >>), pipes, environment variable expansion, bounded command history, SIGINT (Ctrl+C) handling, error handling, and the security requirements defined in SRS Section 5.1. Shell scripting, aliasing, job control (fg/bg/jobs), advanced globbing/wildcards, networking, remote execution, and user authentication are out of scope, consistent with the SRS.

References: Simple Shell SRS v1.0 (08-09-2026), Simple Shell Software Architecture and Design (SAD) Specification v1.0 (14-09-2026), POSIX.1-2017 (IEEE Std 1003.1), GNU C Library (glibc) manual.

Definitions: CLI (Command Line Interface), Shell, Built-in command, External command, PATH, PID (Process Identifier), stdin/stdout/stderr, Redirection, Pipe, SIGINT, Zombie process, POSIX, RTM (Requirements Traceability Matrix), SRS (Software Requirements Specification), SAD (Software Architecture and Design). Full definitions are provided in SRS Section 1.4.

2. Test Items

- Command Input Module (CIM) — prompt display and bounded input reading
- Command Parser (CP) — tokenizing, quoting, and redirection/pipe syntax validation
- Built-in Command Module (BCM) — cd, pwd, echo, help, exit, history
- Process Execution Module (PEM) — PATH lookup, fork/exec, foreground wait, background jobs, zombie reaping
- I/O Redirection Module (IRM) — input (<), output (>), and append (>>) redirection
- Pipe Management Module (PMM) — single and multi-stage pipelines
- Environment Variable Module (EVM) — variable expansion and environment propagation to children
- History Module (HM) — bounded in-memory command history
- Signal Handling Module (SHM) — SIGINT (Ctrl+C) handling
- Error Handling Module (EHM) — plain-English error messaging across all modules

3. Features to be Tested

Features mapped to SRS requirement IDs:

- SH-F-001 to SH-F-005: Prompt display, bounded input, tokenizing, quoted arguments, blank-input handling
- SH-F-006 to SH-F-011: Built-in commands (cd, pwd, echo, help, exit, history)
- SH-F-012 to SH-F-015: PATH lookup, fork/exec, foreground wait, command-not-found handling
- SH-F-016 to SH-F-018: Background execution, PID reporting, zombie reaping
- SH-F-019 to SH-F-022: Input, output, and append redirection; redirection error handling
- SH-F-023 to SH-F-024: Single pipe and multi-command pipe chains
- SH-F-025 to SH-F-026: Environment variable expansion and propagation to children
- SH-F-027 to SH-F-028: Bounded command history; exclusion of blank lines
- SH-F-029 to SH-F-031: SIGINT handling, specific error messages, malformed-input handling

- SH-NF-001 to SH-NF-006: Prompt response time, reliability, usability of messages, maintainability, portability, resource efficiency
- SH-SR-001 to SH-SR-006: Bounded buffers, no shell-evaluation of input, OS permission enforcement, no environment disclosure, redirection/pipe syntax validation, control-character handling

4. Features Not to be Tested

- Shell scripting languages, aliasing, and shell functions (out of scope per SRS 1.2)
- Job control (fg / bg / jobs)
- Advanced globbing / wildcard expansion beyond simple pattern matching
- Networking, remote command execution, and inter-machine communication
- User authentication (single-user, single-session tool)
- Persistent history across sessions (designed for, but not mandated, per SAD Section 4.6)

5. Test Approach / Strategy

Levels

- Unit tests (module-level: CIM, CP, BCM, PEM, IRM, PMM, EVM, HM, SHM, EHM)
- Integration tests (module-to-module, e.g., CP → PEM, PEM → IRM/PMM/EVM)
- System tests (end-to-end shell behaviour on a live POSIX-compliant Linux terminal)
- Acceptance tests (instructor / evaluator sign-off against SRS Section 6 acceptance criteria)

Types

- Functional testing (built-ins, external execution, redirection, pipes, environment expansion, history)
- Regression testing (re-run of the full suite after each fix, especially around PEM and CP)
- Performance testing (prompt response time, per SH-NF-001)
- Reliability / stability testing (no crashes across the full negative and functional suite, per SH-NF-002)
- Usability testing (plain-English error and help messages, per SH-NF-003)
- Security testing (input bounding, injection resistance, permission handling — see 5.1)

Entry Criteria

A stable build of the shell compiles cleanly with GCC on the target POSIX-compliant Linux environment(s); test data (sample files, directories, dummy environment variables) is available; the test environment (terminal, GDB, Valgrind) is ready.

Exit Criteria

100% of planned test cases executed; 0 critical (crash-level) defects open; all High-priority functional, non-functional, and security requirements verified; the Requirements Traceability Matrix (Section 13) is complete.

5.1 Security Validation

- Bounded input/argument buffers — verify oversized input is rejected or safely truncated without memory corruption (SH-SR-001)
- Injection resistance — confirm special characters (';', '"', '\$()') are treated as literal text and never passed to system() or an equivalent shell-evaluation call (SH-SR-002)

- OS permission enforcement — verify redirection and execution never bypass file-permission checks (SH-SR-003)
- No environment disclosure — confirm error messages and logs never dump the full environment block (SH-SR-004)
- Redirection/pipe syntax validation — confirm malformed sequences are rejected before any file is opened or process is forked (SH-SR-005)
- Control-character handling — verify embedded null bytes / non-printable characters are handled without undefined behaviour (SH-SR-006)
- Memory-safety verification using Valgrind and GDB during test execution, per the SAD technology stack (Section 3.6)

6. Test Environment

Hardware: Standard development machine with keyboard and text-capable terminal display; no specialised or dedicated hardware required.

Software: Simple Shell v1.0 (compiled with GCC/C11), glibc, at least two POSIX-compliant Linux distributions (e.g., Ubuntu 20.04 LTS and Ubuntu 22.04 LTS) for portability testing (SH-NF-005).

Tools: GCC (build), Make (build automation), GDB (debugging), Valgrind (memory-safety verification), Bash test scripts for automated regression, Git (version control), Jira / GitHub Issues (defect tracking).

Test Data: Dummy files and directories for redirection tests, sample PATH configurations, sample environment variables (HOME, PATH, USER), long/oversized input strings for boundary and security tests, malformed syntax strings (unmatched quotes, trailing pipes).

7. Test Schedule

Milestones:

Milestone	Target Date
Test plan finalised	17-Sep-2026
Test case design	19-Sep-2026
Test environment setup (build + tools)	20-Sep-2026
Unit & integration test execution	21-Sep-2026 to 24-Sep-2026
System test execution	25-Sep-2026 to 27-Sep-2026
Security & performance test execution	28-Sep-2026
Regression pass	29-Sep-2026
Acceptance review / instructor sign-off	30-Sep-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual and scripted, mapped to SRS requirement IDs)
- Test Scripts (Bash-based functional and regression scripts)
- Test Data (sample files, environment configurations, malformed-input fixtures)

- Test Execution Logs (including Valgrind/GDB output for security and reliability tests)
- Defect Reports
- Requirements Traceability Matrix (RTM)
- Test Summary Report

9. Roles and Responsibilities

Testing responsibilities are concentrated with Aarushi Singh and Akshaya M, who lead test planning, test case design, and the majority of test execution and defect verification. Akshay Anil Kumar and Adarsh Shivanand Madli provide development support, environment setup assistance, and review.

Role	Name	Responsibility
QA Lead / Test Lead	Aarushi Singh	Owns the test plan; designs the majority of test cases across all modules; coordinates schedule and defect triage; leads execution of the Functional, Redirection, Pipe, and Security suites.
Senior Test Engineer	Akshaya M	Designs and executes the majority of test cases for Process Management, Environment & History, Error Handling, Performance, and Reliability suites; maintains the RTM and test execution logs; verifies defect fixes.
Developer / Support Test Engineer	Akshay Anil Kumar	Supports environment setup and build verification; assists with a subset of Built-in Commands and Parsing test cases; triages and fixes defects raised by Aarushi and Akshaya.
Developer / Support Test Engineer	Adarsh Shivanand Madli	Supports test data preparation and portability testing across the two target Linux distributions; assists with a subset of Command Input/Parsing test cases; fixes assigned defects.
Course Instructor	Prof. Pavithra	Reviews test results, approves the Test Plan, and signs off on release/acceptance readiness.

10. Risks and Mitigation

Risk	Mitigation
Buffer overflow from oversized or malformed input	Bounded, fixed-size input/argument buffers with explicit length checks; verified via Security suite (TC-SEC-01) and Valgrind.
Zombie process accumulation from unreaped background jobs	waitpid() with WNOHANG reaping verified under a background-job stress test (TC-PROC-03, TC-RES-01).
Command injection via shell metacharacters	Confirm execvp() is used directly (never system()) via code review and Security suite testing (TC-SEC-02).
Limited availability of two distinct POSIX-compliant Linux environments for portability testing	Use cloud VMs or containers to provision two target distributions early in the schedule.

Risk	Mitigation
Delay in a stable build from development	Request early smoke builds; run smoke tests before full suite execution begins.
Uneven workload if only two team members drive most testing	QA Lead (Aarushi) and Senior Test Engineer (Akshaya) prioritise High-priority requirements first; Akshaya and Adarsh absorb overflow and support tasks to keep the schedule on track.

11. Assumptions & Dependencies

- The host system provides a POSIX-compliant kernel and a standard C runtime library (glibc).
- A properly configured PATH environment variable is available for locating external commands during testing.
- The underlying file system correctly enforces file and directory permissions.
- At least two POSIX-compliant Linux distributions are available for portability testing (SH-NF-005).
- GCC, GDB, and Valgrind are installed and available on the test machine(s).
- The build delivered for testing corresponds to the module structure defined in the SAD (CIM, CP, BCM, PEM, IRM, PMM, EVM, HM, SHM, EHM).

12. Suspension & Resumption Criteria

Suspend testing if:

- The build fails to compile or crashes (segmentation fault) on more than 30% of planned test cases.
- The test environment (terminal/VM) is unavailable for more than 4 hours.
- A High-priority security defect (e.g., buffer overflow or command injection) is found that blocks safe continued testing.

Resume testing if:

- Blocking defects are resolved and verified by Aarushi or Akshaya.
- The test environment is restored and a smoke test passes.

13. Test Case Management & Traceability

The Requirements Traceability Matrix (RTM), maintained in the SRS (Section 8) and supplemented by the requirement-to-module mapping in the SAD (Section 3.8), ensures every SRS requirement is mapped to at least one test case. Representative examples:

Req ID	Requirement (short)	Module(s)	Test Case(s)
SH-F-001	Display prompt	CIM	TC-CLI-01
SH-F-003 / SH-F-004	Tokenize / quoted arguments	CP	TC-PARSE-01, TC-PARSE-02
SH-F-006	'cd' built-in	BCM	TC-BLTIN-CD-01
SH-F-012 to SH-F-015	PATH lookup, fork/exec, wait, not-found error	PEM, EHM	TC-EXEC-01 to TC-EXEC-03

Req ID	Requirement (short)	Module(s)	Test Case(s)
SH-F-016 to SH-F-018	Background execution, PID, reaping	PEM	TC-PROC-01 to TC-PROC-03
SH-F-019 to SH-F-022	Input/output/append redirection	IRM	TC-REDIR-01 to TC-REDIR-03
SH-F-023 / SH-F-024	Single pipe / multi-pipe chain	PMM	TC-PIPE-01, TC-PIPE-02
SH-F-025 / SH-F-026	Env. variable expansion / propagation	EVM	TC-ENV-01
SH-F-027 / SH-F-028	Bounded history / blank-line exclusion	HM	TC-HIST-01, TC-HIST-02
SH-F-029	SIGINT handling	SHM	TC-SIG-01
SH-F-030 / SH-F-031	Error messages / malformed input	EHM, CP	TC-ERR-01, TC-ERR-02
SH-NF-001	Prompt response time	CIM	TC-PERF-01
SH-NF-002	No crashes under test suite	All Modules	TC-REL-01
SH-SR-001 to SH-SR-006	Security requirements	CIM, CP, PEM, IRM, EVM, EHM	TC-SEC-01 to TC-SEC-06

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed
- % passed / failed
- Defect density (defects per module)
- Defect aging (time open)
- Requirement coverage (% of SH-F/SH-NF/SH-SR requirements with at least one passing test case)

Reports:

- Daily execution status, maintained primarily by Aarushi Singh and Akshaya M
- Final Test Summary Report, compiled for instructor review and sign-off

15. Approvals

Role	Name	Signature / Date
QA Lead	Aarushi Singh	
Senior Test Engineer	Akshaya M	
Dev / Support Test Engineer	Akshay Anil Kumar	
Dev / Support Test Engineer	Adarsh Shivanand Madli	
Course Coordinator	Prof. Pavithra	